

Lecture 08: Callbacks, Advanced Array Methods, and Data Structures

Advanced Callbacks:

A complete, standalone function can be passed as an argument (a callback) into another higher-order function. This promotes extreme modularity and reusability.

```
function add(num1,num2){
  return num1 + num2;
}

function sub(num1,num2){
  return num1 - num2;
}

function mul(num1,num2){
  return num1 * num2;
}

function calculator(num1,num2,operation){
  const result = operation(num1,num2);
  console.log(`Result: ${result}`);
}

calculator(2,4,mul);

calculator(10,2,function(a,b){
  return a/b;
});
```

Building a Custom Higher-Order Function:

To understand how built-in JavaScript methods work under the hood, Custom methods can be injected into core JavaScript objects by modifying their prototype.

```
const arr = [1, 3, 5, 4, 2, 7, 9, 8, 6];

Array.prototype.sorting = function(callback) {
  for(let i = 0; i < this.length; i++) {
    for(let j = 0; j < this.length - 1; j++) {
      if(callback(this[j], this[j+1])) {
        let temp = this[j];
        this[j] = this[j+1];
        this[j+1] = temp;
      }
    }
  }
}

arr.sorting((a, b) => a > b);
console.log(arr);
```

Note: Attaching custom methods directly to a single array instance (without using Array.prototype) can cause the function itself to appear during certain iterations. Extending the prototype ensures the method is safely shared across all array instances.

The *this* Keyword:

The *this* keyword acts as a dynamic reference that points to the **current object** executing the code. When extending `Array.prototype`, *this* represents the specific array calling the method.

Built-in Higher-Order Array Methods:

Modern JavaScript provides optimized, built-in higher-order functions for iterating and manipulating arrays.

1. `forEach()`

Executes logic on every element. It accepts a callback with three arguments: the current element, its index, and the full array.

```
const arr = [10, 20, 8, 32, 16];

arr.forEach((element, index, originalArray) => {
  console.log(`Index: ${index}, Value: ${element}`);
});
```

2. `filter()`

Returns a completely new array containing only the values that evaluate to true against the callback condition.

```
const arr = [-10, 2, 8, 30, 17]

const answer = arr.filter((num) => num > 10);
console.log(answer);
```

3. `map()`

Returns a completely new array where every element has been modified by the callback.

```
const arr = [1, 2, 3, 4, 5];

const newArr = arr.map((num) => num * 5);
console.log(newArr);
```

4. `reduce()`

Condenses an array into a single value. It requires an accumulator to track the running total, the current num, and an initial starting value.

```
const arr = [10, 20, 30, 40, 50];

const sum = arr.reduce((accumulator, num) => {
  return accumulator + num;
}, 0);

console.log(sum);
```

Modern ES6 Data Structures:

1. The Set Object

A collection of strictly unique values. A Set will automatically remove duplicate entries.

```
const numbersArray = [1, 2, 3, 3, 4, 2, 5];

const numbersSet = new Set(numbersArray);
console.log(numbersSet);
```

Core Methods:

Method	Description
<code>.add(value)</code>	Appends a new element with a specified value to the end of the Set.
<code>.has(value)</code>	Returns a boolean asserting whether an element is present in the Set.
<code>.delete(value)</code>	Removes the specified element from the Set.
<code>.clear()</code>	Removes all elements from the Set object.

2. The Map Object

A collection of Key-Value pairs, similar to a standard Object. However, Map allows keys of any data type (unlike regular Objects, which restrict keys to Strings or Symbols) and maintains insertion order.

```
const userMap = new Map();
```

```
userMap.set("name", "Rohit");  
userMap.set("age", 20);  
userMap.set(true, "Is Logged In");
```

```
console.log(userMap.get("name"));
```

Core Methods:

Method	Description
<code>.set(key, value)</code>	Adds or updates an element with a specified key and a value.
<code>.get(key)</code>	Returns a specified element from the Map object.
<code>.has(key)</code>	Returns a boolean indicating whether an element with the specified key exists.
<code>.delete(key)</code>	Removes the specified element from the Map by key.
<code>.clear()</code>	Removes all elements from the Map object.

Real-World Application: Method Chaining

Higher-order array methods can be chained together to perform complex data extraction in a highly readable format.

```
const products = [  
  { id: 1, name: "Laptop", category: "Electronics", price: 1200, inStock: true },  
  { id: 2, name: "Smartphone", category: "Electronics", price: 800, inStock: false },  
  { id: 3, name: "The Hobbit", category: "Books", price: 25, inStock: true },  
  { id: 4, name: "Dune", category: "Books", price: 28, inStock: false },  
  { id: 5, name: "Coffee Maker", category: "Appliances", price: 150, inStock: false },  
  { id: 6, name: "Blender", category: "Appliances", price: 80, inStock: true }  
];
```

```
const availableProducts = products  
  .filter(goods => goods.inStock).map(goods => ({  
    name: goods.name,  
    category: goods.category,  
    price: goods.price  
  }));
```

```
console.log(availableProducts);
```